

# SystematicRadicals

Exact radical reconstruction for indexed algebraic numbers

Splitting fields, embedded automorphisms, Kummer towers,  
and practical resolvent methods

Technical article and reference implementations

Prepared for Vladimir Reshetnikov

8 September 2026 • Version 0.1

## Abstract

An exact algebraic number may possess a radical expression even when a computer algebra system leaves its indexed root representation unchanged. This report develops a systematic reconstruction architecture: cheap, exactly verified structural reductions first, followed by a finite splitting-field and Kummer-tower construction. The general method recovers actual embedded field automorphisms, rather than assuming that an abstract permutation group is already labeled by the chosen complex roots. It adjoins only the prime-order roots of unity needed for a prime-index subgroup chain, constructs nonzero eigengenerators by finite trace searches, and determines every principal-root branch by exact algebraic equality. A positive result comes with a checkable straight-line tower certificate.

The accompanying Python implementation was executed and tested; a Wolfram Language reference port and unexecuted kernel tests are also supplied. Mathematical completeness is proved under terminating exact-arithmetic subroutines and unrestricted finite resources. This is not a practical all-degree performance guarantee: default limits and computer algebra failures produce an explicit inconclusive status. The original sextic and quintic, three selected octics from the supplied notebook, branch-sensitive examples, and forced runs of the general backend are treated separately.

**Validation boundary.** Python results in this release are executable results, recorded in the archive. The Wolfram connector returned HTTP 404; no Wolfram kernel execution was available. Static delimiter checks are not kernel validation. The mathematical construction, implemented code, and observed computational coverage are deliberately distinguished.

## Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1 Problem and output contract</b>                             | <b>3</b>  |
| 1.1 Domain and selected root . . . . .                           | 3         |
| 1.2 What the package does not optimize . . . . .                 | 3         |
| <b>2 Audit of the supplied notebook</b>                          | <b>4</b>  |
| <b>3 Exact algebraic infrastructure</b>                          | <b>4</b>  |
| 3.1 An embedded primitive element . . . . .                      | 5         |
| 3.2 Library operations and normalization . . . . .               | 5         |
| <b>4 Practical reductions with exact acceptance</b>              | <b>5</b>  |
| 4.1 Centering and power reduction . . . . .                      | 5         |
| 4.2 Generalized reciprocal reduction . . . . .                   | 6         |
| 4.3 Direct Dickson recognition . . . . .                         | 6         |
| 4.4 Functional decomposition . . . . .                           | 7         |
| 4.5 Pair-sum resolvents and small coefficient fields . . . . .   | 7         |
| <b>5 Why the Galois group is the right obstruction</b>           | <b>7</b>  |
| <b>6 A finite cyclotomic base that suffices</b>                  | <b>8</b>  |
| <b>7 Recovering actual embedded automorphisms</b>                | <b>8</b>  |
| <b>8 A prime-index subnormal chain</b>                           | <b>9</b>  |
| <b>9 Finite Kummer eigengenerators and exact branches</b>        | <b>9</b>  |
| 9.1 The bounded trace search . . . . .                           | 10        |
| 9.2 Rational bases and straight-line formulas . . . . .          | 10        |
| 9.3 The branch is part of the certificate . . . . .              | 11        |
| <b>10 Certificates, verification, and completeness</b>           | <b>11</b> |
| 10.1 A positive certificate . . . . .                            | 11        |
| 10.2 Algorithmic completeness versus software behavior . . . . . | 11        |
| <b>11 Worked reconstruction of the original examples</b>         | <b>12</b> |
| 11.1 The sextic: a cubic followed by a quadratic . . . . .       | 12        |
| 11.2 The quintic: a coupled fifth-root formula . . . . .         | 13        |
| 11.3 Three octics selected from the notebook . . . . .           | 13        |
| 11.4 A genuine negative case . . . . .                           | 14        |
| <b>12 Implementation interfaces and reproducibility</b>          | <b>14</b> |
| 12.1 Python: the executed implementation . . . . .               | 14        |
| 12.2 Wolfram Language: the unexecuted reference port . . . . .   | 15        |
| 12.3 Executed tests and bounded benchmarks . . . . .             | 16        |
| <b>13 Engineering consequences and next developments</b>         | <b>17</b> |
| <b>A Archive layout and rebuild instructions</b>                 | <b>18</b> |
| <b>B General algorithm in pseudocode</b>                         | <b>19</b> |

# 1 Problem and output contract

The starting point is a selected algebraic number, not merely the request to solve an equation. A representative Wolfram Language input is

```
Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2]
```

The original discussion [1] supplies this sextic and a solvable quintic. The supplied notebook explores coefficient fields, projections, subset sums of conjugates, and simplification. The present implementation makes the search reproducible and adds a general constructive backend.

Official documentation states that ToRadicals handles degree at most four and can miss radical expressions that exist in higher degree [2]. Consequently, unchanged output is not a test of nonsolvability. Conversely, replacing a root by a plausible numerical radical is not a solution unless the selected embedding has been identified exactly.

## 1.1 Domain and selected root

The primary domain is  $f \in \mathbb{Q}[x]$  together with one exact root  $\alpha \in \mathbb{C}$ . The Wolfram entry point accepts a single exact algebraic expression whose minimal polynomial over  $\mathbb{Q}$  can be computed. The Python entry point takes a univariate rational polynomial and a zero-based SymPy root index. Algebraic coefficients, symbolic parameters, and approximate coefficient recognition are not part of that Python interface. Repeated factors and reducible input polynomials are allowed, but the algorithm first passes to the minimal polynomial of the selected root.

Indices must not be confused across systems. For the real roots in the original examples, Wolfram indices 2, 5 become Python indices 1, 4. Complex-root orderings need not agree. A cross-system transfer of a complex root must identify the embedding, not merely subtract one from an index.

**Definition 1.1** (Strict radical expression). A strict radical expression is generated from rational constants and  $i$  using finite sums, products, and rational powers wherever the expressions are defined. Powers use the principal complex convention. Other branches are represented by explicit roots of unity. In particular, the final syntax may not contain `Root`, `CRootOf`, `AlgebraicNumber`, trigonometric functions, or exponentials.

Division is multiplication by a negative integer power, and  $i = (-1)^{1/2}$ . The expression

$$\zeta_m = (-1)^{2/m} \quad (m > 1), \quad \zeta_1 = 1 \quad (1)$$

is a permitted representation of the distinguished primitive  $m$ th root of unity. Writing  $\cos(2\pi/m)$  instead would fail the output grammar even though that number is algebraic.

There are four distinct outcomes. Success means that a strict radical representation of the selected number has been certified by exact CAS operations. `NotSolvable` means that an exact Galois calculation has proved nonsolvability over  $\mathbb{Q}$ . `Inconclusive` covers a finite search miss, a resource bound, or a failed backend operation. `InvalidInput` rejects an unsupported domain or malformed option. A timeout is never promoted to a negative theorem.

## 1.2 What the package does not optimize

The task is reconstruction, not optimal denesting. A complete construction can be much larger than the smallest expression for  $\alpha$ . This release does not minimize the number of radicals, their nesting

depth, coefficient height, or the size of an expanded expression. Real input is allowed to produce a complex radical formula whose value is exactly real. A demand that all intermediate radicals be real would be a different problem.

## 2 Audit of the supplied notebook

The archive contains a notebook of 986737 bytes. Inert extraction identified 220 Input cells. Its SHA-256 and the relevant source locations are recorded in `validation/notebook_audit.md`; the notebook was inspected as text, not evaluated. The extractor `tools/inspect_notebook.py` handles common box forms and visibly retains unsupported ones. It is not a general Wolfram parser, and its transcription is not an executable replacement notebook.

The most consequential observations are summarized below. Line numbers are physical lines in the original notebook source, at the start of the cells.

| Line     | Definition                                 | Role and limitation                                                                                 |
|----------|--------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 21       | <code>radicalDepth</code>                  | A complexity measure that also counts sine and cosine; not a strict radical-syntax checker.         |
| 65, 126  | <code>findExtension, findExtension3</code> | Eliminate unknown coefficients of quadratic or cubic factors to find a promising coefficient field. |
| 182      | <code>project</code>                       | Search real and imaginary projections for lower algebraic degree.                                   |
| 252, 315 | <code>factor, solve</code>                 | Factor over an extension and choose a root using context-dependent tests.                           |
| 375, 436 | <code>roots, fuzz</code>                   | Enumerate conjugates; guess fields from high-precision random subset sums.                          |
| 560      | <code>simplify</code>                      | Repeated simplification, private complexity helpers, and special-case exclusions.                   |

The central mathematical idea is good: a root can become a low-degree solution after adjoining an appropriate coefficient field. But a quadratic factor does not imply that its coefficient field is quadratic, and some solvable equations require an odd prime radical step. A search limited to quadratic or cubic factor shapes is not complete.

Numerical recognition is useful for proposing candidates. It should be followed by exact equality with the original algebraic number, not just a small residual, a matching decimal approximation, or an unqualified `PossibleZeroQ` selection. The new code never accepts a candidate on a floating-point tolerance. Numerical values are used only to order a finite candidate list.

The notebook also contains history references, evolving definitions, unfinished inputs, and private helper names, including a misspelled private complexity helper occurrence. These are reasons to rewrite the workflow as a clean package rather than mechanically extract and run every input cell. No claim is made that all 220 cells constitute independent problems or that every large notebook example was solved in this release.

## 3 Exact algebraic infrastructure

Let  $f$  now denote the monic irreducible minimal polynomial of  $\alpha$ . This reduction matters: the polynomial

$$(x - 3)(x^5 - x - 1)$$

has a rational root even though its quintic factor is nonsolvable. A solver that tests the Galois group of the entire supplied polynomial before identifying the selected factor would reject a perfectly valid request.

### 3.1 An embedded primitive element

For a finite number field  $E \subset \mathbb{C}$ , choose an actual embedded primitive element  $\theta$  and its irreducible monic polynomial  $m(t) \in \mathbb{Q}[t]$ . Write  $D = \deg m$ . Computation takes place in

$$R = \mathbb{Q}[t]/(m(t)), \quad t \mapsto \theta. \quad (2)$$

An element is stored as a polynomial of degree below  $D$  with rational coefficients. Products and compositions are reduced modulo  $m$ ; equality is equality of reduced coefficient vectors. If  $b_1, \dots, b_s$  are a known rational basis of a subfield, coordinates of  $a$  are found by solving

$$[[b_1] \ \cdots \ [b_s]] c = [a] \quad (3)$$

over  $\mathbb{Q}$ , and the returned equality is checked exactly.

The embedding is essential. An abstract isomorphism between two number fields need not send the intended root to the intended conjugate. In the Python practical path, equality is checked by expressing a candidate in the specified embedded target field and requiring the coordinate polynomial to be exactly  $t$ . In Wolfram Language the direct practical check is `RootReduce[candidate - target] == 0`. Neither criterion is replaced by equality of minimal polynomials.

### 3.2 Library operations and normalization

The implementation uses standard exact primitives: minimal polynomials, primitive elements, polynomial factorization over number fields, and rational linear algebra. The Wolfram port explicitly requests `ToNumberField[values, All]`; the documentation distinguishes this smallest common field from the potentially larger automatic choice [3]. Coefficients are converted using the documented `AlgebraicNumberPolynomial` operation [4].

The Python implementation uses SymPy number-field and polynomial operations [5]. Polynomial coefficient domains are normalized to  $\mathbb{Q}$  before certificate comparison. This is not cosmetic: serialization may reconstruct an integral-coefficient polynomial with domain  $\mathbb{Z}$  rather than  $\mathbb{Q}$ , even though it represents the same field element. A process-transfer regression test checks this case.

## 4 Practical reductions with exact acceptance

The first layer aims to avoid a splitting field altogether. Every proposed formula must pass both the strict syntax test and exact selected-root equality. The methods below are finite shortcuts, not an alternative proof of universal coverage.

### 4.1 Centering and power reduction

For a monic degree- $n$  polynomial with coefficient  $a_{n-1}$ , set

$$h = -a_{n-1}/n, \quad q(y) = f(y + h).$$

The coefficient of  $y^{n-1}$  vanishes. If the greatest common divisor of the positive exponents appearing in  $q$  is  $g > 1$ , then  $q(y) = Q(y^g)$ . After obtaining a radical root  $b$  of  $Q$ , all lifts are

$$x = h + \zeta_g^k \text{pr}(b^{1/g}), \quad 0 \leq k < g. \quad (4)$$

The notation  $\text{pr}$  emphasizes the principal root; the actual program uses ordinary rational powers and explicit unity factors. If a recursive auxiliary polynomial is reducible, its rational factors are treated separately. A depth limit bounds only this shortcut search.

## 4.2 Generalized reciprocal reduction

Define the Dickson polynomials by

$$D_0(y, a) = 2, \quad D_1(y, a) = y, \quad D_k(y, a) = yD_{k-1}(y, a) - aD_{k-2}(y, a). \quad (5)$$

Induction gives the Laurent identity

$$D_k(u + a/u, a) = u^k + (a/u)^k. \quad (6)$$

For a monic polynomial  $q(x) = \sum_{j=0}^{2m} c_j x^j$  with nonzero constant term, look for a rational  $a \neq 0$  such that

$$q(x) = x^m Q(x + a/x). \quad (7)$$

The top coefficient forces  $c_0 = a^m$ . Therefore only the rational  $m$ th roots of  $c_0$  need be tried, including both signs when appropriate. A candidate is

$$Q(y) = c_m + \sum_{k=1}^m c_{m+k} D_k(y, a). \quad (8)$$

Indeed, substitution yields coefficient pairs  $c_{m-k} = a^k c_{m+k}$ . The code checks the entire polynomial identity (7), rather than relying only on the constant term.

If  $Q(b) = 0$ , the two lifts solve  $x^2 - bx + a = 0$ :

$$x = \frac{b \pm \sqrt{b^2 - 4a}}{2}. \quad (9)$$

A prior centering shift is then restored. The choice  $a = -1$  is as important as  $a = 1$ ; it explains the original sextic.

## 4.3 Direct Dickson recognition

A different use of (6) recognizes

$$q(y) = D_n(y, a) + c.$$

For  $n > 4$ , the coefficient of  $y^{n-2}$  determines  $a = -[y^{n-2}]q/n$ . The full identity is again checked. Setting  $y = u + a/u$  reduces the equation to

$$T^2 + cT + a^n = 0, \quad T = u^n. \quad (10)$$

Choose a nonzero solution  $T$  and let  $u = \text{pr}(T^{1/n})$ . All candidate roots are

$$y_k = \zeta_n^k u + \frac{a}{\zeta_n^k u}, \quad 0 \leq k < n. \quad (11)$$

The branches are coupled: choosing the two  $n$ th roots independently would destroy the product constraint. Conversely, every root  $y$  lifts through  $u^2 - yu + a = 0$ ; replacing  $u$  by  $a/u$  exchanges the two solutions of (10). Thus a nonzero chosen  $T$  supplies all roots with the appropriate multiplicities.

#### 4.4 Functional decomposition

When  $f = A \circ B$  over  $\mathbb{Q}$ , first obtain radical roots  $b$  of  $A$ , then solve  $B(x) = b$ . The present shortcut uses rational polynomial decomposition and only performs the final relative solve when  $\deg B \leq 4$ . Coefficients of that small equation are themselves radical expressions. It does not pretend that a decomposition routine over rational coefficients is already a general decomposition algorithm over arbitrary algebraic coefficient fields.

#### 4.5 Pair-sum resolvents and small coefficient fields

Let the roots of square-free monic  $f$  be  $r_1, \dots, r_n$ . Define

$$R_2(t) = \prod_{i < j} (t - r_i - r_j).$$

It is a rational polynomial invariant under all permutations of the roots. A direct resultant calculation gives

$$\text{Res}_x(f(x), f(t-x)) = 2^n f(t/2) R_2(t)^2. \quad (12)$$

To prove it, write the resultant as  $\prod_{i,j} (t - r_i - r_j)$  and separate the diagonal terms  $i = j$ . The off-diagonal terms occur twice.

Equation (12) makes the notebook's subset-sum idea exact for pairs. Divide out the diagonal factor, factor over  $\mathbb{Q}$ , and halve every factor multiplicity to obtain the monic square root. Distinct pairs can have equal sums, so repeated factors of  $R_2$  must be retained. The test  $x^4 - 1$  specifically exercises this collision case.

For each irreducible factor of  $R_2$  of degree two, three, or four, try its embedded roots  $b$ . Express  $b$  in radicals, factor  $f$  over  $\mathbb{Q}(b)$ , and solve any relative factors of degree at most four. Each coefficient is first converted to a rational polynomial in the same embedded generator  $b$ , and then that generator is replaced by its radical expression. A factor of small degree is useful even when its coefficients have much larger absolute degree after expansion.

Both implementations contain this shortcut, restricted to input degree at most twelve. Its restriction to small pair-sum fields is intentional: a failure to find such a field is not evidence of nonsolvability. The complete backend uses neither this degree cap nor a random search for subset sums.

### 5 Why the Galois group is the right obstruction

For irreducible  $f \in \mathbb{Q}[x]$ , let  $L$  be its splitting field and  $G = \text{Gal}(L/\mathbb{Q})$ . The classical solvability criterion states that  $f$  is solvable by radicals precisely when  $G$  is a solvable finite group [6, Theorem 3.28]. Applied to an irreducible polynomial, the criterion concerns every selected conjugate, not just a convenient root.

The necessity can be understood constructively. Begin with a finite radical tower containing  $\alpha$ . Adjoin the necessary roots of unity and then, at each stage, adjoin roots of all conjugate radicands to make the extension normal. Once the relevant unity factors are present, an automorphism of one such stage sends each radical generator to a unity multiple of itself. The relative Galois group embeds in a product of cyclic groups and is abelian. Induction produces a solvable normal

extension containing all conjugates of  $\alpha$ . Its quotient  $G$  is solvable. The sufficiency is realized explicitly in Sections 6–9.

This also explains two common mistakes. Prime factorization of  $\deg f$  is not a solvability test, and adjoining a few arbitrarily chosen radical constants need not expose a useful factorization. What is required is a suitable tower of subfields of a normal extension.

## 6 A finite cyclotomic base that suffices

Construct  $L = \mathbb{Q}(r_1, \dots, r_n)$  using exact roots and a primitive element. Then

$$d_L = [L : \mathbb{Q}] = |G|.$$

Set

$$M = \prod_{p|d_L} p, \quad B = \mathbb{Q}(\zeta_M), \quad E = L(\zeta_M), \quad (13)$$

where an empty product is 1. Let  $D = [E : \mathbb{Q}]$  and  $H = \text{Gal}(E/B)$ . Because both  $L$  and  $B$  are normal over  $\mathbb{Q}$ , so is  $E$ .

**Lemma 6.1.** *The group  $H$  is solvable if and only if  $G$  is solvable. Moreover,  $|H|$  divides  $d_L$ , and  $B$  contains a primitive  $p$ th root of unity for every prime  $p$  dividing  $|H|$ .*

*Proof.* Restriction identifies  $H$  with  $\text{Gal}(L/(L \cap B))$ , proving the order divisibility. The field  $B/\mathbb{Q}$  is abelian. Write  $\tilde{G} = \text{Gal}(E/\mathbb{Q})$ . If  $H$  is solvable, then so is  $\tilde{G}$ , since its quotient is  $\text{Gal}(B/\mathbb{Q})$ ; hence its quotient  $G$  is solvable. Conversely, the kernel of  $\tilde{G} \twoheadrightarrow G$  is an abelian subgroup of the cyclotomic Galois group. Solvability of  $G$  implies solvability of  $\tilde{G}$  and therefore of  $H$ . Finally,  $p \mid |H|$  implies  $p \mid M$ , and  $\zeta_M^{M/p}$  is a primitive  $p$ th root in  $B$ .  $\square$

Only distinct primes are needed in  $M$ , not every prime power dividing  $d_L$ . Cyclic prime-power quotients will be refined into prime-index steps. One has the useful, but potentially enormous, size bound

$$D \leq d_L \varphi(M) \leq n! \varphi(M). \quad (14)$$

The construction may adjoin roots of unity that turn out not to be needed for the selected root. This is a deliberate tradeoff for a transparent finite algorithm.

## 7 Recovering actual embedded automorphisms

Choose  $E = \mathbb{Q}(\theta)$  and use the quotient representation (2). Since  $E/\mathbb{Q}$  is normal, the minimal polynomial of  $\theta$  splits completely in  $E$ :

$$m(X) = \prod_{j=1}^D (X - a_j(\theta)), \quad a_j(t) \in \mathbb{Q}[t]_{<D}.$$

Every root  $a_j(\theta)$  defines a unique  $\mathbb{Q}$ -automorphism  $\sigma_j$  with  $\sigma_j(\theta) = a_j(\theta)$ . There are no missing embeddings and no repeated ones in characteristic zero. Notice that it is  $m$ , of degree  $D$ , that is factored here; merely listing the  $n$  roots of  $f$  does not list every automorphism of its splitting field.



If  $z(t)$  represents  $\zeta_M$ , retain exactly the indices satisfying

$$z(a_j(t)) \equiv z(t) \pmod{m}. \quad (15)$$

They are the elements of  $H$ . The implementation checks  $|H|\varphi(M) = D$ .

For any field element  $v(t)$ , the action is  $\sigma_j(v) = v(a_j(t)) \bmod m$ . Composition is therefore computable with rational polynomial arithmetic:

$$(\sigma_i \sigma_j)(\theta) = a_j(a_i(\theta)). \quad (16)$$

The program builds an exact multiplication table, finds the identity image  $t$ , verifies closure, and finds inverses. These are genuine automorphisms of the chosen embedded field. An abstract permutation group returned by a small-degree routine is useful for a solvability diagnosis, but cannot be substituted here without an additional certified labeling of its action.

## 8 A prime-index subnormal chain

The derived series is formed using exact table multiplication. For a subgroup  $J$ , compute

$$J' = [J, J] = \langle aba^{-1}b^{-1} : a, b \in J \rangle.$$

The sequence  $H, H', H'', \dots$  eventually reaches the identity or stabilizes at a nontrivial subgroup. The latter gives NotSolvable.

Suppose it reaches the identity. To obtain one prime-index step for a nontrivial solvable subgroup  $J$ , start  $K = J'$ . Enumerate the elements  $g$  of  $J$ ; replace  $K$  by  $\langle K, g \rangle$  whenever this larger subgroup is still proper in  $J$ . The final  $K$  contains  $J'$ , and hence is normal in  $J$ . It is maximal proper: an element rejected earlier would already generate  $J$  with the then smaller subgroup, so it remains rejected after later enlargements. The quotient  $J/K$  is a nontrivial simple abelian group, thus cyclic of prime order.

Repeating yields

$$H = H_0 \triangleright H_1 \triangleright \dots \triangleright H_s = 1, \quad [H_i : H_{i+1}] = p_i \text{ prime}. \quad (17)$$

Only consecutive normality is required. The chain need not consist of subgroups normal in all of  $H$ . Fixed fields give

$$B = E^{H_0} \subset E^{H_1} \subset \dots \subset E^{H_s} = E,$$

and each consecutive extension is cyclic of degree  $p_i$ .

This table-based procedure is elementary and intentionally unoptimized. It does not rely on a database of solvable transitive groups, and it works for solvable groups with mixed prime orders. Its cost can still be severe when  $|H|$  is large.

## 9 Finite Kummer eigengenerators and exact branches

Consider one step  $K \triangleleft J$  of prime index  $p$ , and let  $F = E^J$ ,  $F' = E^K$ . Choose  $\sigma \in J$  whose image generates  $J/K$ . By Lemma 6.1, the number  $\zeta = \zeta_M^{M/p}$  lies in  $F$ . Classical cyclic-extension theory says that such an extension is generated by a radical [6, Proposition 5.27]. The following gives the precise finite search used by the code.

## 9.1 The bounded trace search

For  $0 \leq k < D$ , compute

$$u_k = \sum_{g \in K} g(\theta^k), \quad (18)$$

$$r_k = \sum_{j=0}^{p-1} \zeta^{-j} \sigma^j(u_k). \quad (19)$$

All operations are in  $\mathbb{Q}[t]/m$ . Test  $r_k \neq 0$  exactly, and choose the first nonzero value.

**Proposition 9.1.** *At least one of the  $D$  values  $r_k$  is nonzero. Every nonzero such value  $r$  satisfies*

$$r \in F', \quad \sigma(r) = \zeta r, \quad r^p \in F, \quad F' = F(r).$$

*Proof.* The trace map  $v \mapsto \sum_{g \in K} g(v)$  maps  $E$  onto  $F'$ : on an invariant element it is multiplication by  $|K|$ , which is invertible in characteristic zero. Since  $1, \theta, \dots, \theta^{D-1}$  span  $E$  over  $\mathbb{Q}$ , the  $u_k$  span  $F'$  over  $\mathbb{Q}$ .

On  $F'$ , let  $P = \sum_{j=0}^{p-1} \zeta^{-j} \sigma^j$ . The distinct field automorphisms  $1, \sigma, \dots, \sigma^{p-1}$  are linearly independent as maps  $F' \rightarrow F'$ . Therefore  $P$  is not the zero map, so it cannot vanish on every  $u_k$ . Normality of  $K$  ensures that  $\sigma$  preserves  $F'$ , and  $\sigma^p$  acts trivially on it. Reindexing (19) gives  $\sigma(r) = \zeta r$ . The element  $r^p$  is fixed by both  $K$  and  $\sigma$ , which generate  $J$ , so it belongs to  $F$ . Finally, the  $p$  elements  $r, \zeta r, \dots, \zeta^{p-1} r$  are distinct because  $r \neq 0$ . Thus  $r$  has degree  $p$  over  $F$ , and generates  $F'$ .  $\square$

No random coefficients or success-probability assumptions enter this search. A vanishing trace or projection merely advances  $k$ ; failure for all  $D$  powers would contradict the proposition and is reported as an implementation or backend failure, not as nonsolvability.

## 9.2 Rational bases and straight-line formulas

Start with the rational basis

$$1, \zeta_M, \dots, \zeta_M^{\varphi(M)-1}$$

of  $B$ . Alongside its exact field vectors, store the symbolic list  $1, Z, \dots, Z^{\varphi(M)-1}$ . At a step, use (3) to express  $a = r^p$  in the current basis. The resulting symbolic polynomial  $A(Z, R_1, \dots, R_{i-1})$  is the next radicand.

If  $\beta_1, \dots, \beta_d$  is the current rational basis, the next one is

$$\{\beta_\ell r^j : 0 \leq j < p, 1 \leq \ell \leq d\}. \quad (20)$$

It is a basis because  $1, r, \dots, r^{p-1}$  is an  $F$ -basis of  $F'$ . The symbolic basis is enlarged using a fresh variable  $R_i$  in the same order. Eventually the basis has  $D$  elements. One last rational linear solve expresses  $\alpha$  as a polynomial in  $Z, R_1, \dots, R_s$ .

Separating field vectors from formula syntax avoids repeated expansion of nested radicals during the expensive algebra. The final substitution can still be large; the intermediate straight-line tower is retained in the certificate.

### 9.3 The branch is part of the certificate

Knowing only  $r^p = a$  does not identify  $r$ . Compute the actual embedded numbers  $a(\theta)$  and  $r(\theta)$ . Among the  $p$  candidates

$$\zeta_M^{(M/p)b} \text{pr}(a(\theta)^{1/p}), \quad b = 0, \dots, p-1, \quad (21)$$

choose the one exactly equal to  $r(\theta)$ . Such a  $b$  exists because both numbers are roots of  $X^p - a$ , and  $r \neq 0$ . Numerical approximations may put a likely candidate first, but every acceptance is an exact field embedding or RootReduce equality.

The program then records the syntactic rule

$$R_i = \zeta_M^{(M/p_i)b_i} \text{pr}(A_i(Z, R_1, \dots, R_{i-1})^{1/p_i}).$$

By induction, substituting earlier rules gives the correct embedded value at every stage. Finally substitute  $Z = (-1)^{2/M}$ . The answer is in the strict radical grammar without concealing any unity factors in trigonometric functions. No generally invalid law such as  $(ab)^{1/p} = a^{1/p}b^{1/p}$  is used to simplify branches.

## 10 Certificates, verification, and completeness

### 10.1 A positive certificate

A complete result stores  $\theta, m$ , the field degrees,  $M$ , the vectors for  $\zeta_M$  and  $\alpha$ , automorphism images, the subgroup chain, and a straight-line formula. Each radical step stores its prime  $p$ , radicand, branch integer, and field polynomial representing the generator.

The independent positive checker need not trust the subgroup search. It checks the irreducible field representation and the selected embeddings; then it checks each equality  $r_i^{p_i} = A_i$  in the quotient ring and each branch equation (21) in the embedded number field. It finally checks the coordinate formula for the target and that substituting the stored rules gives the returned expression. This is an inductive proof of the positive result using exact CAS arithmetic.

During construction, all branch equalities are already checked. Python also performs a second, cheaper ring-identity check before returning. The public `verify_tower(result, target)` defaults to checking embeddings and branches again. Its optional `check_branches=False` mode checks only the algebraic consistency of the recorded formulas; it must not be advertised as an independent proof of the selected embedding.

Practical results instead store the target and its minimal polynomial and are accepted by direct exact equality. Negative results are supported by an exact group calculation. This release does not include a separate general negative-certificate verifier. In particular, a list of derived subgroup orders by itself is not a self-contained proof; it must be backed by the actual automorphism action or the trusted Galois routine.

### 10.2 Algorithmic completeness versus software behavior

**Theorem 10.1** (Finite exact reconstruction). *Assume that exact root isolation, minimal polynomials, primitive elements, number-field factorization, embedded equality, and rational linear algebra terminate*

correctly. With no resource bounds, the general construction returns a radical representation of every algebraic number over  $\mathbb{Q}$  that is expressible by radicals, and detects nonsolvability otherwise.

*Proof.* Reduction to the selected irreducible minimal polynomial is finite. The splitting field  $L$ , cyclotomic compositum  $E$ , and automorphism list are finite. Lemma 6.1 reduces the solvability question to  $H$ . Its finite derived series either stabilizes nontrivially, proving nonsolvability, or reaches the identity. In the latter case the finite subgroup construction gives (17). Proposition 9.1 supplies a nonzero generator in a search through at most  $D$  powers at each step. Exact coordinates and exact branch choices produce a valid radical formula. There are at most  $\log_2 |H|$  steps. The terminal fixed field is all of  $E$ , hence contains  $\alpha$ .  $\square$

The theorem is not a claim that every concrete call to a large CAS succeeds within available time or memory. The source delegates substantial exact algebra to its host library. A host exception is surfaced as `Inconclusive`. Finite defaults deliberately weaken the unconditional termination contract into a bounded attempt. The Python `method="complete"` and Wolfram `Method->"Complete"` options bypass every practical shortcut; they are the entry points corresponding to the theorem when all external limits are removed.

The full field can have factorial degree in  $n$ . The program constructs a quadratic-size automorphism multiplication table and performs repeated polynomial compositions and rational linear solves, with potentially large coefficient heights. Expanded radical output may be far larger than the input. Therefore no useful polynomial bound in the input degree is claimed. An efficient general production solver would need substantially better Galois, relative-field, and expression-management infrastructure.

## 11 Worked reconstruction of the original examples

The two input polynomials below come from the original discussion [1]. The calculations and exact checks here are also reproduced by the accompanying code. The substitutions are not presented as newly discovered solutions to that discussion; their systematic recognition and exact branch treatment are the point.

### 11.1 The sextic: a cubic followed by a quadratic

Let

$$f(x) = x^6 + x^4 - x^3 - x^2 - 1.$$

A direct expansion gives

$$f(x) = x^3 \left( (x - x^{-1})^3 + 4(x - x^{-1}) - 1 \right). \quad (22)$$

Thus the reciprocal parameter is  $a = -1$ , and the lower polynomial is  $q(y) = y^3 + 4y - 1$ . Let

$$C = \left( \frac{9 + \sqrt{849}}{18} \right)^{1/3}, \quad b = C - \frac{4}{3C}.$$

Here  $C$  is positive real. To verify  $b^3 + 4b - 1 = 0$ , put  $v = -4/(3C)$ . Then  $Cv = -4/3$  and  $C^3 + v^3 = 1$ , so  $(C + v)^3 + 4(C + v) = 1$ . Since  $q'(y) = 3y^2 + 4 > 0$ , this is its unique real root. Its sign is positive because  $q(0) = -1$ .

The desired positive real root of  $f$  is

$$\boxed{\alpha = \frac{b + \sqrt{b^2 + 4}}{2}} = 1.1306854454620405872 \dots \quad (23)$$

The other real lift is negative; the nonreal cubic roots cannot arise as  $x - 1/x$  for real  $x$ . Hence (23) is precisely Wolfram root index 2, or Python index 1.

For comparison with the notebook's extension idea, the pair-sum resolvent contains  $t^3 + 4t - 1$  as a factor, and over  $\mathbb{Q}(b)$  one obtains

$$f(x) = (x^2 - bx - 1) \cdot (x^4 + bx^3 + (b^2 + 2)x^2 - bx + 1).$$

The full pair resolvent is

$$R_2(t) = (t^3 + 4t - 1)(t^{12} - t^9 + 8t^8 - 2t^7 - t^6 + 4t^5 + 18t^4 + 9t^3 + 2t^2 - 1).$$

These are exact polynomial identities, not numerical discoveries requiring an assumed tolerance.

## 11.2 The quintic: a coupled fifth-root formula

For

$$g(x) = 5x^5 - 25x^3 + 25x + 6 = 5D_5(x, 1) + 6,$$

put

$$T = \frac{-3 + 4i}{5}, \quad u = \text{pr}(T^{1/5}).$$

Since  $T^2 + \frac{6}{5}T + 1 = 0$ , equation (6) yields

$$\boxed{\beta = u + u^{-1}} = 1.8070599950854245021 \dots \quad (24)$$

All five roots are supplied by  $u\zeta_5^k + (u\zeta_5^k)^{-1}$ . For interpretation only, write  $T = e^{i\vartheta}$  with  $0 < \vartheta < \pi$ . The roots are  $2 \cos((\vartheta + 2\pi k)/5)$ ; the principal  $k = 0$  choice has the smallest angular distance to 0 and is the largest real root. Thus it is Wolfram index 5, or Python index 4. The trigonometric interpretation is not used as the returned radical syntax.

The implementation's representation uses a single fifth root and its reciprocal, which preserves branch coupling automatically. Testing all five indices is important: a formula that happens to select the largest real root does not establish that the root-index handling is correct in general.

## 11.3 Three octics selected from the notebook

Denote by  $A, B, C$  the following polynomials:

$$\begin{aligned} A(x) &= x^8 + 8x^7 + 20x^6 + 8x^5 - 36x^4 - 48x^3 - 15x^2 + 2x + 1, \\ B(x) &= x^8 + 8x^7 + 10x^6 - 52x^5 - 131x^4 - 28x^3 + 100x^2 + 32x + 16, \\ C(x) &= x^8 + 8x^7 + 16x^6 - 4x^5 - 68x^4 - 28x^3 + 49x^2 + 14x + 1. \end{aligned}$$

The selected root is the least real one in each case. Centering at  $-1$  makes  $A$  and  $B$  even, so each reduces to a quartic in a square. For  $C$ , the pair-sum resolvent contains

$$(t^2 + 4t + 1)^2.$$

Taking the corresponding quadratic field exposes quartic factors. One returned form of the least root is

$$-1 - \frac{\sqrt{3}}{2} - \sqrt{\sqrt{3} + \frac{9}{4} + \frac{\sqrt{15}}{2} \sqrt{4\sqrt{3} + 7}}. \quad (25)$$

All three selected-root equalities were checked exactly in Python. Turning off the pair-resolvent shortcut makes the practical method inconclusive on  $C$ ; this is an explicit regression test for the distinction between a search miss and a negative theorem.

## 11.4 A genuine negative case

For  $h(x) = x^5 - x - 1$ , the Python low-degree Galois calculation reports a nonsolvable group of order 120. There is also a short independent structural explanation. Modulo 3,  $h$  is irreducible; modulo 2,

$$h(x) = (x^2 + x + 1)(x^3 + x^2 + 1).$$

The factorizations are square-free. The associated Frobenius cycle type  $(2)(3)$  yields a transposition on taking its cube. Irreducibility makes the action transitive, and transitive prime-degree actions are primitive. A primitive permutation group containing a transposition is the full symmetric group: the graph whose edges are all conjugates of that transposition has a group-invariant component partition, hence is connected, and its edge transpositions generate the full symmetric group. Thus the group is  $S_5$ , which is nonsolvable. This negative conclusion is not inferred from the failure of a radical search.

# 12 Implementation interfaces and reproducibility

## 12.1 Python: the executed implementation

The implementation is `python/systematic_radicals.py`. From the archive root, a minimal example is

```
import sys
sys.path.insert(0, "python")
import sympy as s
from systematic_radicals import solve_radical, verify_tower
x = s.Symbol("x")
r = solve_radical(x**6+x**4-x**3-x**2-1, 1,
                  method="practical")
assert r.status == "Success"
print(r.expression)

t = solve_radical(x**3-2, 0, method="complete")
assert t.status == "Success"
assert verify_tower(t, s.CRootOf(x**3-2, 0))
```

The optional arguments are `max_field_degree=128`, `max_depth=12`, and `pair_resolvent=True`. The "automatic" mode runs shortcuts and then the general backend; "practical" stops after the finite shortcut layer; "complete" forces the splitting-field construction. Python's small-degree negative shortcut uses the documented Galois routine for degrees at most six [5]; the general backend does not call that routine.

The direct function has no wall-clock limit. The wrapper `solve_with_timeout`, with a seconds argument, runs a fresh spawned process, returns an inconclusive status on timeout, and terminates the worker. It should be called from a saved script under a main guard. The CLI provides an immediate bounded alternative:

```
python python/systematic_radicals.py \
    "5*x**5-25*x**3+25*x+6" --index 4 --method practical
```

The CLI parser is for trusted local expressions. It is not a sandbox for untrusted strings. To remove mathematical limits, use the direct complete method with `max_field_degree=None`, without an external timeout. A degree cap is checked after primitive-element construction and is not a bound on memory already spent constructing that element.

## 12.2 Wolfram Language: the unexecuted reference port

The package is loaded without replacing any built-in definitions:

```
Get["Wolfram/SystematicRadicals.wl"];
a = Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2];
r = RadicalSolve[a, Method -> "Practical"];
If[r["Status"] === "Success",
  RadicalVerify[a, r["Expression"]]]

b = Root[6 + 25 # - 25 #^3 + 5 #^5 &, 5];
SystematicToRadicals[b, Method -> "Practical"]
```

`RadicalSolve` returns an association. `SystematicToRadicals` returns the expression on success and a `Failure` otherwise. `RadicalExpressionQ`, `RadicalVerify`, `RadicalTowerVerify`, and `PairSumResolvent` are public helpers.

The default options are:

```
Method -> "Automatic", TimeConstraint -> 60,
"MaxFieldDegree" -> 128, "MaxDepth" -> 12,
"PairResolvent" -> True
```

Unlike the Python path, it first tries built-in `ToRadicals`, and it has no SymPy low-degree Galois diagnostic. Complete mode bypasses shortcuts. Both `TimeConstraint` and `"MaxFieldDegree"` accept `Infinity`. An infinite bound removes a limit, not the underlying computational difficulty.

The port uses documented algebraic operations and a custom finite-group multiplication table. Nevertheless, static inspection cannot establish Wolfram evaluation semantics or version compatibility. The included `Wolfram/tests.wlt` and `Wolfram/run-tests.wls` must be executed on a real kernel. No Wolfram benchmark figures are asserted in this report.

### 12.3 Executed tests and bounded benchmarks

The preparation environment used Python 3.13.5 and SymPy 1.14.0. Exact regression tests cover both original examples, all five Dickson branches, nonreal binomial branches, repeated and reducible inputs, inexact and parametric rejection, a nonsolvable quintic, resource-limit statuses, resolvent multiplicity collisions, corrupted certificates, and process transfer of certificate polynomials. Separate finite-group table tests exercise  $S_3$ ,  $S_4$ ,  $A_5$ , and  $S_5$ , including nonsolvable derived-series stabilization; these are unit tests, not claims to have built those large splitting fields in the benchmark. Forced complete tests prevent the practical layer from concealing a broken general construction.

Actual test counts and elapsed times appear in the generated validation record below and in `validation/pytest.log`. The benchmark includes spawn overhead and was not designed as a controlled performance study. It should not be read as a comparison of CAS implementations.

**Recorded Python test run.** 61 passed, 0 failures or errors, 0 skipped; 51.72 seconds in the recorded run. The test suite and machine-readable JUnit record are included in the archive.

| Input and Python index         | Path           | Status       | $D$ | Seconds |
|--------------------------------|----------------|--------------|-----|---------|
| Original sextic, $i = 1$       | Reciprocal     | Success      | –   | 2.31    |
| Original quintic, $i = 4$      | Dickson        | Success      | –   | 1.64    |
| $x^5 - x - 1$ , $i = 0$        | Galois test    | Not solvable | –   | 1.22    |
| $x^2 - 2$ , $i = 1$            | General        | Success      | 2   | 1.05    |
| $x^3 - 2$ , $i = 0$            | General        | Success      | 6   | 1.61    |
| $x^4 - 2$ , $i = 1$            | General        | Success      | 8   | 26.06   |
| $x^4 + 1$ , $i = 0$            | General        | Success      | 4   | 1.24    |
| $\Phi_5(x)$ , $i = 0$          | General        | Success      | 4   | 1.43    |
| Notebook octic $A$ , $i = 0$   | Power          | Success      | –   | 1.38    |
| Notebook octic $B$ , $i = 0$   | Power          | Success      | –   | 1.18    |
| Notebook octic $C$ , $i = 0$   | Pair resolvent | Success      | –   | 1.79    |
| $x^5 - 2$ (automatic), $i = 0$ | Power          | Success      | –   | 1.23    |
| $x^5 - 2$ (forced), $i = 0$    | General        | Inconclusive | –   | 30.02   |

Table 1: Recorded bounded runs.  $D = [E : \mathbb{Q}]$  is shown only when the general field was constructed. Indices are zero-based SymPy indices. The last row reached its 30-second limit; no nonsolvability conclusion follows.

For every small successful complete benchmark, the independent verifier was run with embedding and branch checking enabled. In particular,  $x^4 - 2$  exercises a nonabelian splitting-field group and a three-step prime tower. The cubic exercises a nontrivial cyclotomic base and a cubic Kummer step. The cyclotomic quartics exercise complex target embeddings.

Forced complete reconstruction of  $x^5 - 2$  exceeded the 30-second worker limit in this environment. Automatic reconstruction of the same input succeeded through the power shortcut. This deliberately included contrast shows why the mathematically complete backend is a fallback rather than a replacement for structure recognition. It also places a concrete limit on claims of practical coverage in this release.



## 13 Engineering consequences and next developments

The dominant opportunity is to avoid constructing a large absolute splitting field. An optimized system would use relative number fields, resolvents for a targeted solvable chain, and exact embeddings only where needed. A fast Galois routine is valuable, but its abstract output must be connected to the actual roots before it can drive the eigengenerator construction. Guessing that labeling from floating-point order is not an acceptable replacement.

The current basis update and subgroup enumeration are straightforward rather than optimized. Better implementations can retain sparse field representations, reuse multiplication matrices, cache subfield coordinate maps, choose smaller eigengenerators, and postpone expansion of the final straight-line program. These are efficiency improvements, not changes to the correctness argument.

A second direction is a stronger portable certificate format with a small independent arithmetic checker for both positive and negative results. The present positive checker remains a CAS-level verifier. Its explicit embedding and branch data are a useful starting point for a proof-assistant formalization, but the report does not claim to supply one.

Finally, denesting and presentation should be a separate, verified postprocessing layer. One may rank alternate subgroup chains, conjugate generators, reciprocal transformations, or factorizations by expression size, but every simplification must preserve the selected algebraic number. Repeated `FullSimplify` calls or private complexity functions are not part of the completeness proof. The essential separation is: *discover a candidate efficiently, construct a general answer when necessary, and certify the exact selected value in either case.*

## A Archive layout and rebuild instructions

```
SystematicRadicals/  
  README.md           usage, scope, validation boundary  
  LICENSE             new code only  
  requirements.txt     tested SymPy version; pytest for tests  
  Makefile            article, tests, benchmark targets  
  article/  
    SystematicRadicals.tex  
    SystematicRadicals.pdf  
    validation-table.tex  
  python/systematic_radicals.py  
  Wolfram/  
    SystematicRadicals.wl  
    tests.wlt          not executed in this environment  
    run-tests.wls  
  examples/examples.py  
  examples/examples.wl  
  tests/test_systematic_radicals.py  
  tools/benchmark.py  
  tools/build_validation_table.py  
  tools/inspect_notebook.py  
  validation/  
    benchmark.json  
    benchmark.log  
    pytest.xml  
    pytest.log  
    notebook_audit.md  
    static_checks.json
```

Run the Python tests and bounded benchmark from the archive root:

```
python -m pytest -q tests  
python tools/benchmark.py validation/benchmark.json
```

Build the article by running pdfLaTeX twice in its directory, or use `make article`. The article source is self-contained apart from the accompanying generated validation table. The original notebook is not redistributed or relicensed.

## B General algorithm in pseudocode

```

INPUT: exact selected algebraic number alpha
f := monic minimal polynomial of alpha over Q
L := Q(all exact roots of f)
M := product of distinct primes dividing [L:Q]
z := principal  $(-1)^{(2/M)}$ , with z=1 for M=1
E := L(z); choose E=Q(theta), modulus m
A := all roots of m inside Q[t]/m
H := automorphisms theta -> a(theta) in A that fix z
build exact multiplication table for H

if the derived series of H stabilizes nontrivially:
    return NotSolvable

basis := [1, z, ..., z^(phi(M)-1)]
syntax := [1, Z, ..., Z^(phi(M)-1)]
J := H
while J is not trivial:
    choose K normal in J with prime index p
    choose sigma generating J/K
    zeta := z^(M/p)
    for k=0, ..., [E:Q]-1:
        u := sum_{g in K} g(theta^k)
        r := sum_{j=0}^{p-1} zeta^(-j) sigma^j(u)
        stop at the first exactly nonzero r
    A := rational coordinates of r^p in basis, applied to syntax
    choose b in {0, ..., p-1} by exact embedded branch equality
    record R := z^(M/p*b) * principal(A^(1/p))
    basis := [beta*r^j : j=0, ..., p-1, beta in basis]
    syntax := [S*R^j : j=0, ..., p-1, S in syntax]
    J := K

F := rational coordinates of alpha in basis, applied to syntax
substitute the recorded radical rules and Z=z into F
verify the tower and strict radical syntax
return Success, expression, certificate

```

The order of basis elements and symbolic monomials is identical. This small implementation detail is part of the certificate invariant; a change of basis order without the corresponding formula change corrupts the answer.

## References

- [1] Vladimir Reshetnikov, with answers and comments by contributors. *Why is ToRadicals[] not able to handle all cases? Is there a workaround?* Mathematica Stack Exchange, question 34011, 2013. <https://mathematica.stackexchange.com/questions/34011>. Accessed 8 September 2026.
- [2] Wolfram Research. *ToRadicals*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToRadicals.html>. Accessed 8 September 2026.
- [3] Wolfram Research. *ToNumberField*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToNumberField.html>. Accessed 8 September 2026.
- [4] Wolfram Research. *AlgebraicNumberPolynomial*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html>. Accessed 8 September 2026.
- [5] SymPy development team. *Number Fields*, SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/numberfields.html>. Accessed 8 September 2026. This release was executed with SymPy 1.14.0.
- [6] J. S. Milne. *Fields and Galois Theory*. Course notes, especially Theorem 3.28 and Proposition 5.27. <https://www.jmilne.org/math/CourseNotes/FT.pdf>. Accessed 8 September 2026. Cited pages use the printed numbering, 45 and 74.